

รายงานการทดลองที่ 2: Atmel AVR Studio Tutorial

ผู้จัดทำ: 1. นายดรณ์ภพ พิทักษ์กิจไพศาล (6710301007) 2. นายอภิศักดิ์ คงภักดี (6710301009) 3. นายธนัท จงธีรธนโชติ (6710301032)

วิชา: Embedded System Design เสนอ: อาจารย์ศุภาเทพ สวัสดิ์พิศาล ภาคการศึกษา: 1/2569

1. การวิเคราะห์โค้ดการทดลอง (Exercise 1)

Project AssemblerApp1 เป็นการเขียนโปรแกรมภาษา Assembly เบื้องต้นเพื่อบวกลบค่าเลขคณิตและจัดเก็บผลลัพธ์ลงใน SRAM รายละเอียดโค้ดและการวิเคราะห์ผลมีดังนี้:

```
.include "m328pdef.inc" ; นำเข้าไฟล์จำกัดความของชิป ATmega328P
.list ; แสดงผลลัพธ์เครื่องในไฟล์ลิสต์ (.lss)

rjmp main ; กระโดดไปยัง main หลังเปิดเครื่องหรือ Reset

main:
    ldi r16,0x99 ; โหลดค่า 0x99 เข้าเก็บใน Register R16
    sts 0x212,r16 ; เขียนบันทึกค่าใน R16 ลงแรม SRAM ตำแหน่ง 0x212
    ldi r17,0x15 ; โหลดค่า 0x15 เข้าเก็บใน Register R17
    ldi r18,0x20 ; โหลดค่า 0x20 เข้าเก็บใน Register R18
    add r17,r18 ; บวกลบค่า R17 + R18 (0x15 + 0x20) เก็บผล 0x35 ไว้ใน R17
    sts 0x213,r17 ; เขียนบันทึกผลลัพธ์จาก R17 ลง SRAM ตำแหน่ง 0x213

here:
    rjmp here ; วนรอบจบการทำงานปกติอย่างปลอดภัย
```

ผลการจำลองการทำงานและข้อมูลใน SRAM

จากการทดสอบรันโค้ดแบบทีละบรรทัด (Step Into) บนระบบจำลอง Simulator สามารถติดตามพฤติกรรมข้อมูลได้ดังนี้:

- เมื่อคำสั่ง `ldi r16, 0x99` ทำงาน ค่าใน Register R16 จะเปลี่ยนเป็น **0x99**
- เมื่อคำสั่ง `sts 0x212, r16` ทำงาน ค่าสะสมจะถูกบันทึกเขียนตรงไปยัง SRAM Address **0x212**
- เมื่อคำสั่ง `add r17, r18` ทำงาน ค่าใน R17 จะกลายเป็น **0x35** จากสมการผลบวก $0x15 + 0x20 = 0x35$
- เมื่อคำสั่ง `sts 0x213, r17` ทำงาน จะนำคำตอบผลรวม **0x35** ไปเขียนบันทึกลง SRAM Address **0x213**

Address	+0	+1	+2	+3	+4	+5	+6	+7
0x0210	00	00	99	35	00	00	00	00

2. การเขียนโปรแกรมคำนวณและจัดเก็บข้อมูลใน SRAM (Exercise 2)

การทดลองส่วนนี้เป็นการเขียนโปรแกรมคำนวณโจทย์ $0x23 + 0x72 - 0x14$ และจัดเก็บผลลัพธ์ลงใน SRAM Address **0x214** ของชิป ATmega328P:

```
.include "m328pdef.inc"

rjmp main          ; เริ่มทำงานที่ป้ายชื่อ main หลังการ Reset

main:
  ldi r16, 0x23     ; โหลดค่า 0x23 ไปยัง Register R16
  ldi r17, 0x72     ; โหลดค่า 0x72 ไปยัง Register R17
  add r16, r17      ; R16 = R16 + R17 = 0x23 + 0x72 (ผลลัพธ์ได้ 0x95)
  ldi r17, 0x14     ; โหลดค่า 0x14 ไปยัง Register R17 เพื่อนำมาลบ
  sub r16, r17      ; R16 = R16 - R17 = 0x95 - 0x14 (ผลลัพธ์สุดท้ายได้ 0x81)
  sts 0x214, r16    ; จัดเก็บค่าใน R16 (0x81) ลงใน SRAM Address 0x214

here:
  rjmp here        ; วนรอบหยุดการทำงานเพื่อความปลอดภัย
```

ผลการจำลองการทำงานและข้อมูลใน SRAM

จากการวิเคราะห์โค้ดที่ผ่านการรันโปรแกรม Debugger Simulator:

- เมื่อทำงานถึงคำสั่งบวกค่า ผลลัพธ์ใน R16 มีค่าสะสมเป็น **0x95**
- เมื่อทำงานถึงคำสั่งลบค่า ผลลัพธ์สุดท้ายใน R16 จะลดลงเหลือ **0x81** (เทียบเท่า 129 ในระบบฐานสิบ)
- เมื่อคำสั่ง **sts 0x214, r16** ทำงาน ค่าผลลัพธ์ **0x81** จะเขียนบันทึกลงแรม SRAM Address **0x214** โดยตรง

Address	+0	+1	+2	+3	+4	+5	+6	+7
0x0210	00	00	00	00	81	00	00	00

3. ตอบคำถามท้ายบทการทดลอง

คำถามข้อที่ 1: Watch Window คืออะไร? ทำงานอย่างไร?

คำตอบ: Watch Window คือเครื่องมือในโปรแกรม Debugger ที่คอยสืบค้นและแสดงผลค่าของตัวแปรหรือ Register ที่เราสนใจแบบเรียลไทม์ขณะ Debug โดยจะดึงค่าจาก CPU Simulator มาแสดงผลโดยอัตโนมัติเมื่อกด Step รันคำสั่งทีละบรรทัด

คำถามข้อที่ 2: อธิบายการทำงานของคำสั่ง ADD

คำตอบ: คำสั่ง **ADD Rd, Rr** จะบวกข้อมูลจาก Register ต้นทาง **Rr** เข้ากับ Register ปลายทาง **Rd** แล้วนำผลรวมเขียนทับลงใน Register ปลายทาง **Rd** โดยใช้เวลานาน 1 รอบสัญญาณนาฬิกา และส่งผลอัปเดตสถานะของ Flag ต่าง ๆ ใน SREG ทันที

คำถามข้อที่ 3: คำสั่ง Reset ใน Debug menu ทำหน้าที่อะไร?

คำตอบ: คำสั่ง Reset จะดึงค่าของ Program Counter (PC) กลับไปยังจุดเริ่มต้น Address **0x0000** เพื่อเตรียมเริ่มต้นประมวลผลคำสั่งใหม่ทั้งหมดเหมือนพฤติกรรมการกดปุ่ม Reset Hardware จริง แต่จะไม่ล้างข้อมูลใน Register ทัวไปและหน่วยความจำ SRAM